



第六次技术培训

灾害救援 – 第19届智能车大赛

Technical Support VI

- 工程物理系科协：邓浩然
- 2025 年 3 月 26 日

PART 01

控制小车建图（补充讲解）

- 对于机器人，要想实现自主行走就必须解决这三个问题：
- 1.Mapping：我在哪？我们需要给机器人提供一张机器人可以高精度的全局地图。然后机器人要判断当前它的出发点在地图的哪个位置。这个问题在机器人领域的专有名词是“定位”。
- 2.Localization：我要去哪？我们给出目的地在全局地图上的哪个位置。

■ 3.Navigation: 怎么过去?

- 机器人首先要根据起始点位置和全局地图先规划出一条可行走的路线（即不与固定的障碍物碰撞），即我们说的全局路径规划。
- 然后机器人开始沿这个路径行走，在行走的过程中，机器人会实时检测自己前方（局部地图）中是否有全局地图中没有的障碍物（临时障碍物），如果有就避开它，即我们说的局部路径规划。

■ Localization 和 Navigation 是我们在初赛已经完成任务。我们这里主要解决 Mapping。我们这里采用 ROS 内置的 Cartographer 构图算法。

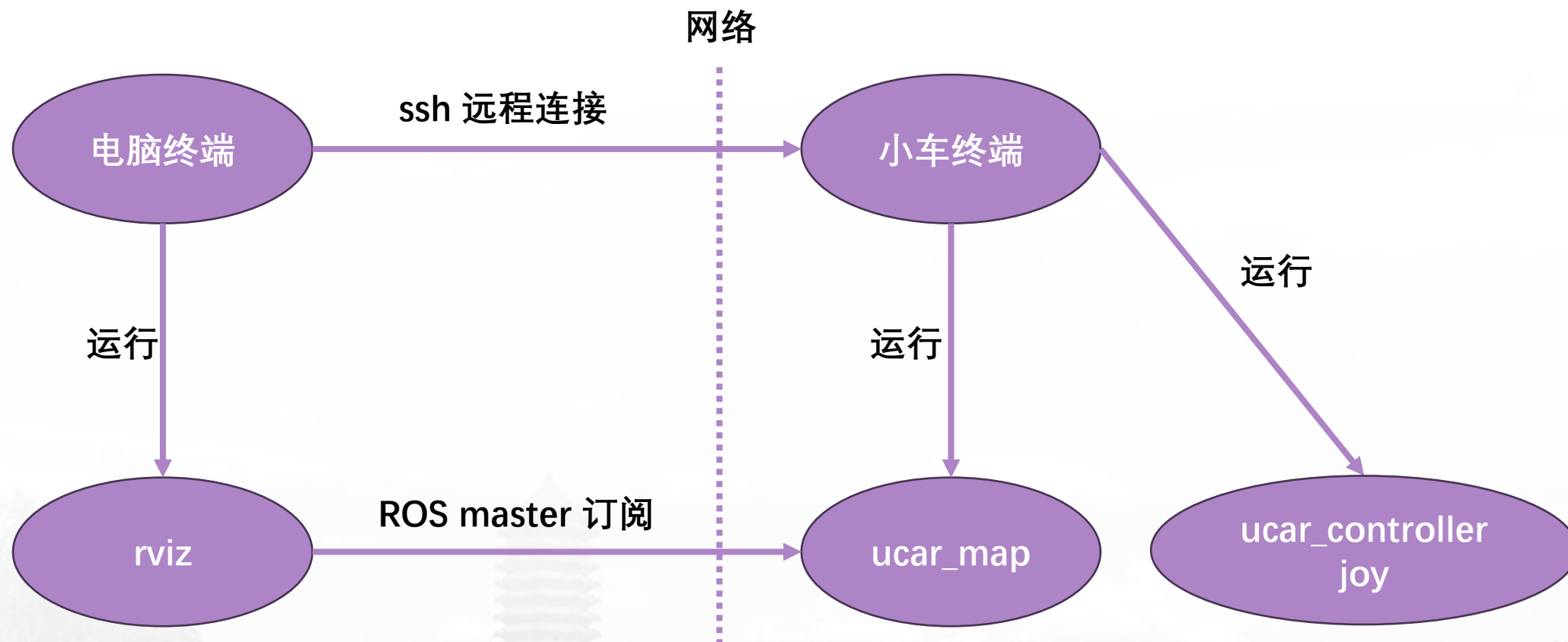
■ 在小车终端输入：

- `roslaunch ucar_map ucar_mapping.launch`

■ 你应当会看到小车的激光雷达开始旋转，代表小车已经开始了激光扫描与构建。

■ 在电脑终端输入：

- `roslaunch rviz rviz`
- 通过刚刚配置好的 ROS master，电脑上的 rviz 可以订阅小车的话题。
- 小车上当然也可以运行 rviz，但是我们使用终端连接小车和电脑，终端无法传输 rviz 的图形界面（vnc 和远程桌面等方式可以做到，但配置相对麻烦）。



- 接下来，我们在 `rviz` 中订阅 `/map` 和 `/laser` 话题：
 - 1. 点击左下角 `Add` 按键，在 `By display type` 中依次选择 `LaserScan` 和 `Map` 插件，点击 `OK` 从而完成添加。
 - 2. 在左侧 `Displays` 区域中，在 `LaserScan` 中的 `Topic` 选项的下拉菜单中选择话题 `/laser/scan`，在 `Map` 中的 `Topic` 选项的下拉菜单中选择话题 `/map`。
- 现在，你应该可以在 `rviz` 中看到地图信息和雷达信息。

- 你可以遥控机器人遍历整个区域，也可以多次重复行走，以保证雷达能够更加准确地探测区域，
- 如果感觉边缘已比较清晰，就可以进行地图的保存了
- 在小车终端执行如下指令：
 - `cd ~/ucar_ws/src/ucar_map/maps/`
 - `roslaunch map_server map_saver --occ 70 --free 20 -f ucar_map_001`
- 此时在 `maps` 文件夹中将会有 `ucar_map_001.pgm` 和 `ucar_map_001.yaml` 两个文件，这些文件即表征了我们刚才建立的地图。

- `roslaunch map_server map_saver --occ 70 --free 20 -f ucar_map_001`
- 第一个参数 `--occ` 为占用阈值 (`threshold_occupied`)
 - 雷达得到的是每个像素被占用的概率
 - 当像素占据的概率大于占用阈值时候认为是完全占据的
- 第二个参数 `--free` 为自由阈值 (`threshold_free`)
 - 当像素占据的概率小于自由阈值时候认为是完全是空的
- 第三个参数为保存的文件名

- 地图配置文件文件 (.yaml) :

```
image: testmap.png    地图图像的路径
resolution: 0.1        地图的分辨率 像素
MIorigin: [0.0, 0.0, 0.0] 初始点位置 在地图上的坐标以及 航偏。航偏是指角度。
occupied_thresh: 0.65   当像素占据的概率大于 0.65 时候认为是完全占据的。
free_thresh: 0.196      当像素占据概率小于 0.196 的时候, 认为完全是空的
negate: 0
```

- 其中参数正是刚才提到的阈值。可以进行调整。

- 图片形式的地图文件 (.pgm) :
- 该文件是对栅格占用状态的数据进行了编码, 利用不同的像素颜色来描述每个栅格的占用状态。
- 在标准配置中, 白像素 (255) 代表未被占据 (自由空间), 黑像素 (0) 代表被占据 (障碍物), 灰像素代表状态未知 (未知区域)。



- 一般来说，判断一个区域是否被占据按照如下占据值来判断：
 - $occ = (255 - color_avg) / 255.0$
- 占据值与占用阈值和自由阈值的大小关系给出栅格的占用、未知、自由三种状态。
- 根据上届选手的经验，因为雷达扫描精度的问题，得到的地图文件可能存在模糊或者偏差，可以通过修改 .pgm 文件（也就是手动绘图）的方式进行调整。
 - 可以使用本地编辑器或者 [Pixilart - Free Online Art Community and Pixel Art Tool](#) 在线编辑 .pgm 文件。
 - 也可以使用 OpenCV 等库进行去除噪点等操作。

PART 02

语音播放

- 首先需要选手下载第六次技术培训的 ucar_ws.zip。
 - 其中的内容不能在虚拟机中运行，而是需要选手之后用 scp 命令拷贝到小车中运行。
 - 本次技术培训讲解其中几个重点模块。
- src/play_sound_demo 文件夹提供了播放 wav 文件的 Python 源代码（注意，这不是功能包，而是段与 ROS 独立的代码）。同学们可通过执行以下指令来播放 wav 文件：
 - `python ~/ucar_ws/src/play_sound_demo/ucar_play_sounds.py`
- 运行指令后、提示输入时，输入 0 并回车，效果为播放 sounds/0.wav，即播报“自强不息，厚德载物”。

- 源代码位于 `~/ucar_ws/src/play_sound_demo/ucar_play_sounds.py`，与播放 wav 有关的主要代码如下：
- 其中 `filepath` 为 wav 文件的路径（绝对、相对均可）。

```
1  #! /usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  import os
4  import playsound
5
6  print("You should put .wav file under `sounds` dir.")
7  while True:
8      try:
9          name = input("file name for sound(without .wav):")
10         filepath = os.path.join(os.path.dirname(__file__), "sounds", name + ".wav")
11         if os.path.exists(filepath):
12             playsound.playsound(filepath)
13     except KeyboardInterrupt:
14         break
```

PART 03

语音输入

- 在赛事资源的 ucar_ws.zip 中，src/xf_mic_asr_offline 文件夹提供了与麦克风阵列有关的功能包。同学们可通过执行以下指令来语音控制小车灯光的位置：
 - `roslaunch xf_mic_asr_offline xf_mic_asr_offline.launch`
- 运行指令后、命令行输出“未设置主麦，请唤醒或设置主麦”时，喊一声“小微小微”，效果为点亮小车顶部 12 个 LED 灯中声源方向的那一盏灯。

- 源代码位于 `~/ucar_Ws/src/xf_mic_asr_offline/src/hid_test_auto.cpp`。这个文件看似有小 1000 行，但其中需要我们修改的内容并不多，我们一一分析。
- 1 ~ 25 行：引用头文件。
- 27 ~ 47 行：声明全局变量。
- 49 ~ 126 行：定义一些辅助函数，如获取文件大小 `FileSize`、`std::string` 和 `std::wstring` 之间的相互转换 `ws2s` 和 `s2ws`、判断字符串是否为整数 `isnumber`、字符串与字符数组拼接 `join`。
 - 不过实话说，写代码的人应该不怎么会 OOP，有些代码写得过于丑陋。

- 128 ~ 391 行：麦克风通讯回调函数 `business_proc_callback`，语音输入的主要逻辑函数，其中与语音输入后执行逻辑有关的核心代码位于 299 ~ 323 行。
- 392 ~ 438 行：显示离线命令词识别结果 `show_result`。
- 440 ~ 520 行：回调函数，获取离线命令词识别结果 `Get_Offline_Recognise_Result`。
- 522 ~ 559 行：回调函数，获取麦克风音频 `Record_Start`。
- 561 ~ 603 行：回调函数，设置麦克风唤醒词 `Set_Awake_Word`。
- 605 ~ 658 行：回调函数，设置亮灯编号 `Set_Led_On`。
- 660 ~ 729 行：回调函数，设置主麦克风 `Set_Major_Mic`。

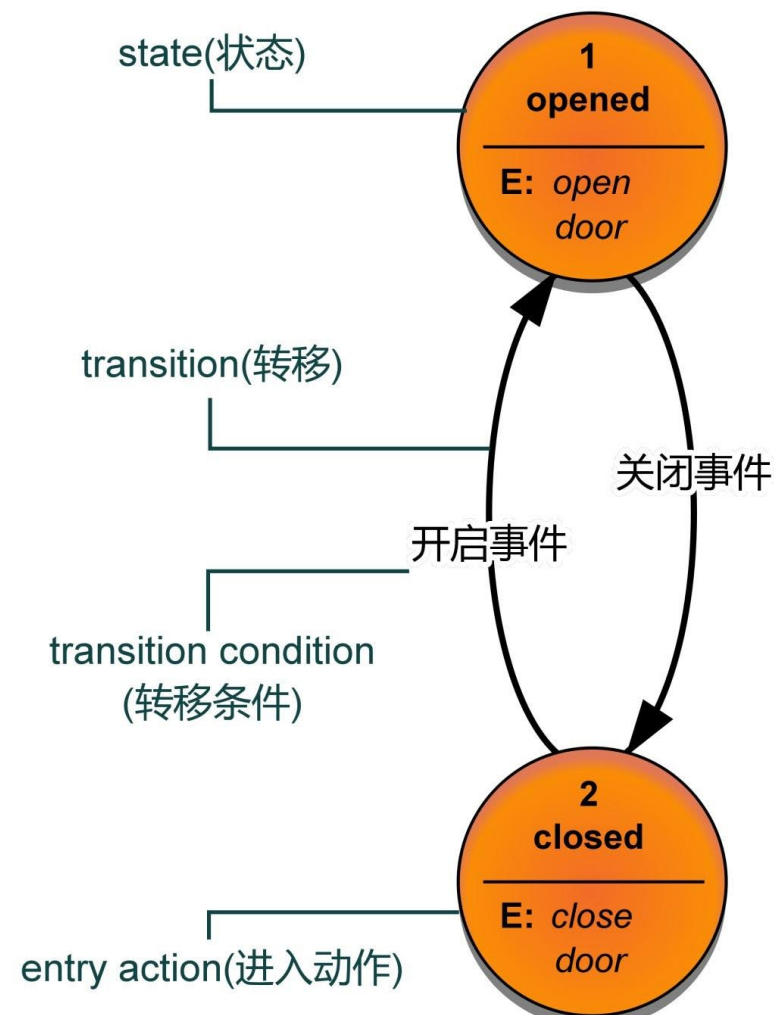
- 731 ~ 757 行：回调函数，获取主麦克风编号 Get_Major_Mic。
- 759 ~ 782 行：回调函数，获取唤醒角度 Get_Awake_Angle。
- 784 ~ 864 行：程序入口 main，其中 794 ~ 799 行发布了与麦克风识别有关的话题，801 ~ 820 行发布了上述回调函数对应的服务。
- 选手只需要重点关注 794 ~ 799 行中 pub_pcm、pub_awake_angle 和 major_mic_pub 在源文件出现的位置，因为它们将麦克风的识别结果发布到了 /mic/awake/angle、/mic/pcm/deno 和 /mic/major_mic 三个话题上。我们后续可以通过订阅这些话题获取麦克风的识别结果。同学们也可以在此基础上发布其他话题。

PART 04

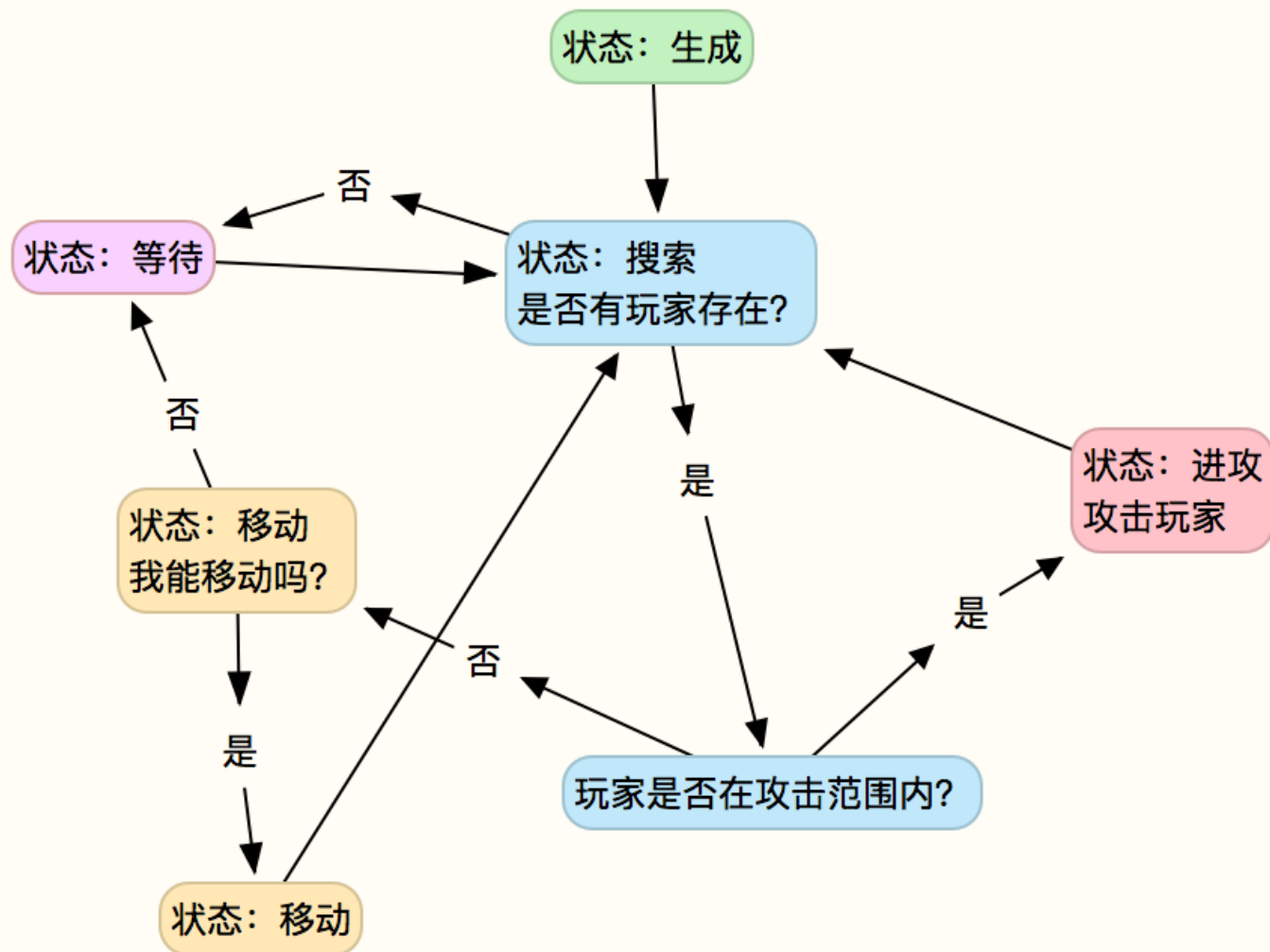
状态机

有限状态机

- 如果选手想要使用更复杂的框架，可以利用状态机实现。
- 一个有限状态机（后文以状态机缩写指代）是表示有限个状态以及在这些状态之间的转移和动作等行为的**数学计算模型**。
- 状态机由 **state（状态）**、**transition（转移）**、**action（动作）**、**transition condition（转移条件）** 组成。



- 一个游戏怪物的状态机如图：



- 状态机是一种抽象的方法，接下来我们需要考虑如何具体实现它。
- 最简单的实现是使用一个变量记录当前状态，再用分支语句根据当前状态和转移条件判断改转移到哪个状态、并修改变量。
- 在程序中每一个状态都是独立的代码块，与其他不同的状态分开独立运行，这么做可以使得调试变得更加方便，同时也更易于增加新的功能。
- 任何语言或环境中你都可以这样实现，而 ROS 中有专门的方法。

ROS中一般使用SMACH来实现状态机，SMACH是一种基于python的强大的、可否压缩的分级状态机库

SMACH 及状态机可视化工具安装：

```
sudo apt-get install ros-melodic-executive-smach  
sudo apt-get install ros-melodic-executive-smach-visualization
```

创建一个状态，以类的形式

切换行号显示

```
1  class Foo(smach.State):  
2      def __init__(self, outcomes=['outcome1', 'outcome2']):  
3          # 你的状态在这里进行初始化  
4  
5      def execute(self, userdata):  
6          # 你的状态在这里运行  
7          if xxxx:  
8              return 'outcome1'  
9          else:  
10             return 'outcome2'
```

向状态机添加状态

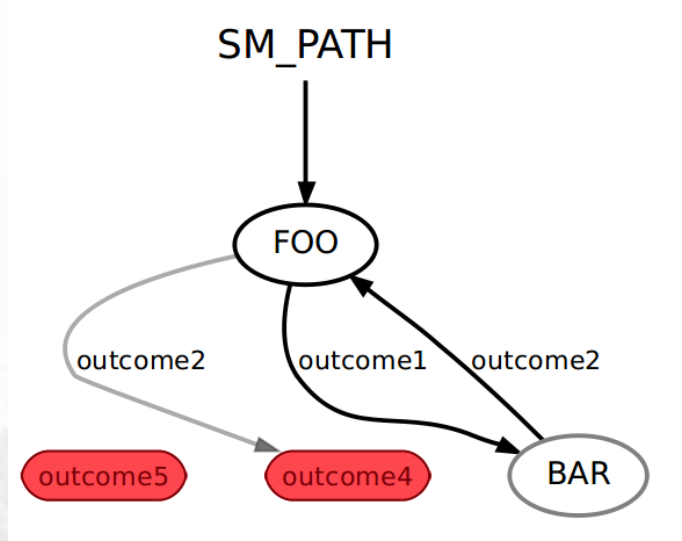
使用`StateMachine`创建一个状态机，且制定状态机执行完后的输出值

```
sm = smach.StateMachine(outcomes=['outcome4', 'outcome5'])
```

使用`add()`方法添加需要的状态到状态机容器，同时需要设置状态间的跳转关系

```
with sm:  
    smach.StateMachine.add('FOO', Foo(),  
        transitions={'outcome1': 'BAR', 'outcome2': 'outcome4'})  
    smach.StateMachine.add('BAR', Bar(),  
        transitions={'outcome2': 'FOO'})
```

状态机的结果：

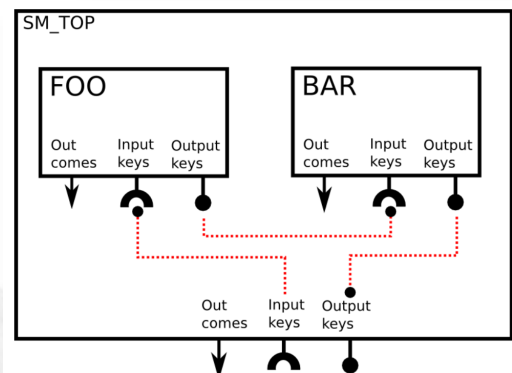


给状态指定输入输出数据

状态的用户数据储存在userdata结构体中，可定义状态输入输出的用户数据
Input_keys和output_keys

状态间传递数据

Remapping, 通过重新映射把数据从
FOO传递到BAR



```
class Foo(smach.State):
    def __init__(self, outcomes=['outcome1', 'outcome2'],
                 input_keys=['foo_input'],
                 output_keys=['foo_output']):
        pass

    def execute(self, userdata):
        # Do something with userdata
        if userdata.foo_input == 1:
            return 'outcome1'
        else:
            userdata.foo_output = 3
            return 'outcome2'
```

```
sm_top = smach.StateMachine(outcomes=['outcome4', 'outcome5'],
                             input_keys=['sm_input'],
                             output_keys=['sm_output'])

with sm_top:
    smach.StateMachine.add('FOO', Foo(),
                           transitions={'outcome1': 'BAR',
                                         'outcome2': 'outcome4'},
                           remapping={'foo_input': 'sm_input',
                                       'foo_output': 'sm_data'})

    smach.StateMachine.add('BAR', Bar(),
                           transitions={'outcome2': 'FOO'},
                           remapping={'bar_input': 'sm_data',
                                       'bar_output1': 'sm_output'})
```

程序框架

```
def main():
    rospy.init_node('smach_example_state_machine')
    # 创建一个状态机
    sm = smach.StateMachine(outcomes=['outcome4'])
    sm userdata.sm_counter = 0 # 定义用户数据
    打开状态机容器
    with sm:
        # 添加状态

    # 创建并启动内部监测服务器
    sis = smach_ros.IntrospectionServer('my_smach_introspection_server', sm, '/SM_ROOT')
    sis.start()

    # 开始执行状态机
    outcome = sm.execute()

    # 等待退出
    rospy.spin()
    sis.stop()

if __name__ == '__main__':
    main()
```

多状态的并行，使用concurrency()

```
# default_outcome: 表示该状态机的默认输出为outcome4
# outcome_map: 设置了状态机同步运行的状态跳转
# 当FOO状态输出为outcome2，且BAR状态输出为outcome1时状态机才会输出outcome5.
sm_con = smach.Concurrence(outcomes=['outcome4', 'outcome5'],
                             default_outcome='outcome4',
                             outcome_map={'outcome5':
                                           { 'FOO': 'outcome2',
                                             'BAR': 'outcome1' }})
```

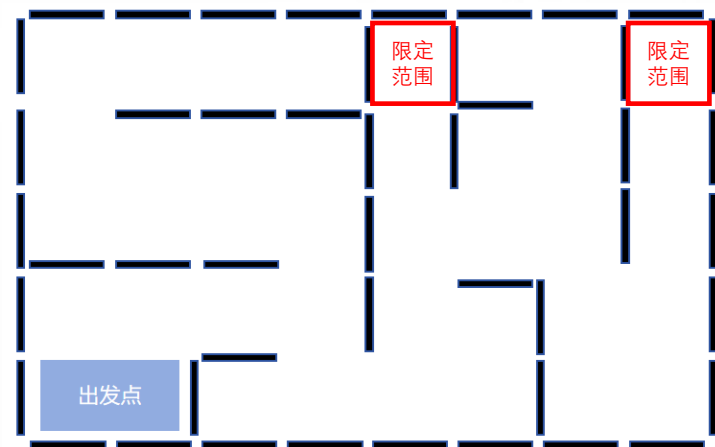
PART -2147483647

决赛规则

决赛规则

1. 智能车经过语音唤醒启动 (10分)
2. 智能车根据辐射信号导航到**较近**的待救人员处 (20分) , 语音播报身份 (10分) , 再返回出发点 (10分)
3. 智能车根据辐射信号导航到**较远**的待救人员处 (20分) , 语音播报身份 (10分) , 再返回出发点 (10分)
4. 打印两待救人员身份及坐标 (5分×2)

- 2、3任务顺序颠倒的, 得2、3两任务原本得分的70%分数
- 总时间最多15min, 超过者不计分
- 到达待救人员处和返回出发点处时**所有轮子都不在限定范围内的不得分**, **一或两个轮子在限定范围内的得该任务一半分数**, **三个轮子及以上得该任务所有分数**。(轮子与地面的接触点在范围内即可)
- 待救人员坐标与实际坐标**误差超过0.1 (不含) 的不得分**, **0.01到0.1之间的得一半分数**, **小于0.01 (不含) 的得满分**
- 待救人员身份判断错误 (语音播报错误或打印错误) 扣5分 → 例: 播报身份+10 播报/打印错误-5 + 误差小于0.01+5= +10
- 各队伍按照总时间 (从语音唤醒到第二次返回出发点) 进行排序, 从用时最少到用时最多线性地在上述任务得分的基础上乘以100%到85%的系数作为最终得分

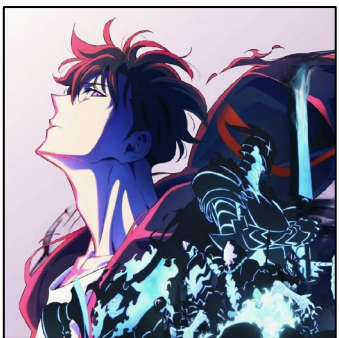


决赛地图示意

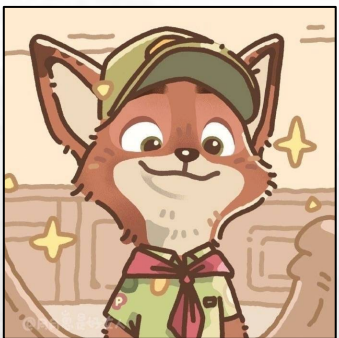
利用图像识别功能识别待救人员的身份



← A 第三十三届工物科协主席



← B 第三十二届工物科协主席



← C 第三十一届工物科协主席

1. 待救人员的身份有三种可能，分别记为A、B、C，分别对应一张图片（右图所示）。
2. 比赛时每个待救人员处会贴上其中一张（三选二），小车到达待救人员处后，利用小车配备的摄像头拍摄到的图像，设计算法识别其身份（**后续培训会重点介绍**）。
3. 我们会提供小车实拍得到的图片文件与原图作为训练集。



欢迎交流提问

灾害救援 – 第19届智能车大赛

Feel free to ask questions!

- > 工程物理系科协：邓浩然
- > 2025 年 3 月 26 日